



OData Compact JSON Format Version 4.02

Committee Specification Draft 01

20 August 2026

This stage:

<https://docs.oasis-open.org/odata/odata-compact-json-format/v4.02/csd01/odata-compact-json-format-v4.02-csd01.md> (Authoritative)

<https://docs.oasis-open.org/odata/odata-compact-json-format/v4.02/csd01/odata-compact-json-format-v4.02-csd01.html>

<https://docs.oasis-open.org/odata/odata-compact-json-format/v4.02/csd01/odata-compact-json-format-v4.02-csd01.pdf>

Previous stage:

N/A

Latest stage:

<https://docs.oasis-open.org/odata/odata-compact-json-format/v4.02/odata-compact-json-format-v4.02.md> (Authoritative)

<https://docs.oasis-open.org/odata/odata-compact-json-format/v4.02/odata-compact-json-format-v4.02.html>

<https://docs.oasis-open.org/odata/odata-compact-json-format/v4.02/odata-compact-json-format-v4.02.pdf>

Technical Committee:

[OASIS Open Data Protocol \(OData\) TC](#)

Chairs:

Ralf Handl (ralf.handl@sap.com), [SAP SE](#)

Michael Pizzo (b-mpizzo@microsoft.com), [Microsoft](#)

Editors:

Michael Pizzo (b-mpizzo@microsoft.com), [Microsoft](#)

Hubert Heijkers (hubert.heijkers@nl.ibm.com), [IBM](#)

Related work:

This specification is related to:

- *OData Version 4.02*. Edited by Michael Pizzo, Ralf Handl, and Heiko Theißen. A multi-part Work Product that includes:
 - *OData Version 4.02 Part 1: Protocol*. Latest stage. <https://docs.oasis-open.org/odata/odata/v4.02/odata-v4.02-part1-protocol.html>
 - *OData Version 4.02 Part 2: URL Conventions*. Latest stage. <https://docs.oasis-open.org/odata/odata/v4.02/odata-v4.02-part2-url-conventions.html>
 - *ABNF components: OData ABNF Construction Rules Version 4.02 and OData ABNF Test Cases*. <https://docs.oasis-open.org/odata/odata/v4.02/csd01/abnf/>

- *OData Vocabularies Version 4.0*. Edited by Michael Pizzo, Ralf Handl, and Ram Jeyaraman. Latest stage: <https://docs.oasis-open.org/odata/odata-vocabularies/v4.0/odata-vocabularies-v4.0.html>
- *OData Common Schema Definition Language (CSDL) JSON Representation Version 4.02*. Edited by Michael Pizzo, Ralf Handl, and Heiko Theißen. Latest stage: <https://docs.oasis-open.org/odata/odata-csdl-json/v4.02/odata-csdl-json-v4.02.html>
- *OData Common Schema Definition Language (CSDL) XML Representation Version 4.02*. Edited by Michael Pizzo, Ralf Handl, and Heiko Theißen. Latest stage: <https://docs.oasis-open.org/odata/odata-csdl-xml/v4.02/odata-csdl-xml-v4.02.html>
- *OData JSON Format Version 4.02*. Edited by Michael Pizzo, Ralf Handl, and Heiko Theißen. Latest stage: <https://docs.oasis-open.org/odata/odata-json-format/v4.02/odata-json-format-v4.02.html>
- *OData Data Aggregation Extension Version 4.0*. Edited by Ralf Handl, Hubert Heijkers, Gerald Krause, Michael Pizzo, Heiko Theißen, and Martin Zurmuehl. Latest stage: <https://docs.oasis-open.org/odata/odata-data-aggregation-ext/v4.0/odata-data-aggregation-ext-v4.0.html>
- *OData Extension for Temporal Data Version 4.0*. Edited by Ralf Handl, Hubert Heijkers, Gerald Krause, Michael Pizzo, Heiko Theißen, and Martin Zurmuehl. Latest stage: <https://docs.oasis-open.org/odata/odata-temporal-ext/v4.0/odata-temporal-ext-v4.0.html>

Abstract:

The Open Data Protocol (OData) for representing and interacting with structured content is comprised of a set of specifications. The core specification for the protocol is in OData Version 4.02 Part 1: Protocol. This document extends the core specification by defining representations for OData requests and responses using a compact JSON format.

Status:

This document was last revised or approved by the OASIS Open Data Protocol (OData) TC on the above date. The level of approval is also listed above. Check the “Latest stage” location noted above for possible later revisions of this document. Any other numbered Versions and other technical work produced by the Technical Committee (TC) are listed at <https://groups.oasis-open.org/communities/tc-community-home2?CommunityKey=e7cac2a9-2d18-4640-b94d-018dc7d3f0e2#technical>.

TC members should send comments on this specification to the TC’s email list. Any individual may submit comments to the TC by sending email to Technical-Committee-Comments@oasis-open.org. Please use a Subject line like “Comment on OData Protocol”.

This specification is provided under the [RF on RAND Terms Mode](#) of the [OASIS IPR Policy](#), the mode chosen when the Technical Committee was established. For information on whether any patents have been disclosed that may be essential to implementing this specification, and any offers of patent licensing terms, please refer to the Intellectual Property Rights section of the TC’s web page (<https://www.oasis-open.org/committees/odata/ipr.php>).

Note that any machine-readable content ([Computer Language Definitions](#)) declared Normative for this Work Product is provided in separate plain text files. In the event of a discrepancy between any such plain text file and display content in the Work Product’s prose narrative document(s), the content in the separate plain text file prevails.

Key words:

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “NOT RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in BCP 14 [\[RFC2119\]](#) and [\[RFC8174\]](#) when, and only when, they appear in all capitals, as shown here.

Citation format:

When referencing this specification the following citation format should be used:

[OData-Compact-JSON-Format-v4.02]

OData Compact JSON Format Version 4.02. Edited by Michael Pizzo and Hubert Heijkers. 20 August 2026. OASIS Committee Specification Draft 01. <https://docs.oasis-open.org/odata/odata-compact-json-format/v4.02/csd01/odata-compact-json-format-v4.02-csd01.html>. Latest stage: <https://docs.oasis-open.org/odata/odata-compact-json-format/v4.02/odata-compact-json-format-v4.02.html>.

Notices

Copyright © OASIS Open 2026. All Rights Reserved.

Distributed under the terms of the OASIS [IPR Policy](#).

The name “OASIS” is a trademark of [OASIS](#), the owner and developer of this specification, and should be used only to refer to the organization and its official outputs.

For complete copyright information please see the full Notices section in an Appendix below.

Table of Contents

1 Introduction	
1.1 Glossary	
1.1.1 Definitions of Terms	
1.1.2 Acronyms and Abbreviations	
1.1.3 Document Conventions	
2 Compact JSON Format Design	
2.1 Design Principles	
2.2 Relationship to the OData JSON Format	
3 Requesting the Compact JSON Format	
3.1 Format Parameters	
3.2 Advertising Support	
4 Compact Representations	
4.1 Positional Representation	
4.2 Positional Property List	
4.3 Determining the Positional Property List	
4.4 Grouping of Select Items	
4.5 The Wrapper Object	
4.6 Position Values	
5 Common Characteristics	
5.1 Header Content-Type	
5.2 Message Body	
5.3 Payload Ordering Constraints	
6 Structured Instances	
6.1 Entity	
6.2 Complex Value	
6.3 Collection of Entities	
6.4 Collection of Complex Values	
6.5 Primitive Values	
6.6 Navigation Properties	
6.7 Derived Types	
6.8 Open Types and Dynamic Properties	
6.9 Entity References	
6.10 Media Entities and Stream Properties	
7 Annotations and Control Information	
7.1 Instance Annotations	
7.2 Property Annotations	
7.3 Selected Annotations	
7.4 Control Information	
7.4.1 Control Information: context	
7.4.2 Control Information: count and nextLink	
7.4.3 Control Information: type	
8 Request Payloads	
8.1 Determining the Positional Property List in Requests	
8.2 Message Body of a Request	
8.3 Creating an Entity	

[8.4 Updating an Entity](#)

[8.5 Updating a Collection of Entities](#)

[8.6 Deep Insert](#)

[8.7 Bind Operations](#)

[8.8 Action and Function Parameters](#)

[9 Delta Payload](#)

[9.1 Added or Changed Entity](#)

[9.2 Deleted Entity](#)

[9.3 Added and Deleted Links](#)

[9.4 Nested Delta Payloads](#)

[10 Service Document](#)

[11 Error Response](#)

[12 Batch Requests and Responses](#)

[13 Conformance](#)

[14 Open Issues](#)

[14.1 Cross-Specification Dependencies](#)

[14.2 Naming and Encoding Decisions](#)

[14.3 Constructs Not Yet Specified](#)

[A References](#)

[A.1 Normative References](#)

[A.2 Informative References](#)

[B Safety, Security and Privacy Considerations](#)

[C Acknowledgments](#)

[C.1 Special Thanks](#)

[C.2 Participants](#)

[D Revision History](#)

[E Notices](#)

1 Introduction

The OData protocol is comprised of a set of specifications for representing and interacting with structured content. The core specification for the protocol is in [\[OData-Protocol\]](#); this document is an extension of the core protocol. Representations for OData requests and responses using the JavaScript Object Notation (JSON), see [\[RFC8259\]](#), are defined in [\[OData-JSON\]](#).

This document defines a *compact* JSON format: a lossless, alternative representation of the same information, whose prime aim is to minimize the uncompressed size of OData request and response payloads.

Processing large volumes of JSON is expensive, largely due to its verbosity. Compression on the wire takes most of that inefficiency away during transport, but the sender producing and the receiver consuming the uncompressed payload still deal with the full volume of JSON text. The repetition of property names, once per property per instance, dominates that volume in exactly the payloads where it hurts most: large collections of entities and large collections of complex values.

The approach taken in this format, inspired by traditional rowset interfaces, is to represent a structured instance as a JSON *array* rather than a JSON object, so that a property name is transmitted at most once per payload instead of once per instance. The items of a JSON array are ordered, whereas the name/value pairs of a JSON object are not; this format relies on that ordering to convey, by position, which value belongs to which property.

The information needed to map a position back to a property is already present in every OData payload: the [context URL](#) describes what a payload contains, and has always been designed to allow a payload to be interpreted without knowledge of the request that produced it. This format gives the context URL the additional role of describing *at which position* each property appears.

A compact JSON payload may represent anything an OData JSON payload may represent:

- a [single primitive value](#)
- a [collection of primitive values](#)
- a [single complex value](#) or a [collection of complex values](#)
- a [single entity](#) or [entity reference](#)
- a [collection of entities](#) or entity references
- a [collection of changes](#)
- a [service document](#) describing the top-level resources exposed by the service
- an [error](#).

1.1 Glossary

1.1.1 Definitions of Terms

The following terms are used throughout this document:

- *Positional representation*: the representation of a structured instance as a JSON array whose items are the values of the instance's properties, identified by their position.
- *Positional property list*: the ordered list of properties that a positional representation conveys; item *n* of the array is the value of item *n* of this list. See [section 4.2](#).
- *Wrapper object*: a JSON object that appears where a value would otherwise appear, carrying whatever must be conveyed by name alongside that value — annotations, control information, and properties that are not in the positional property list — together with the value itself. See [section 4.5](#).
- *Compact payload*: a request or response body labeled with the [compact](#) format parameter.

[1.1.2 Acronyms and Abbreviations](#)

[1.1.3 Document Conventions](#)

Keywords defined by this specification use **this monospaced font**.

Some sections of this specification are illustrated with non-normative examples.

Example 1: text describing an example uses this paragraph style

```
Non-normative examples use this paragraph style.
```

All examples in this document are non-normative and informative only.

Examples use the sample model of [\[OData-Protocol\]](#) with an entity type **Customer** having structural properties **ID**, **Name** and **Address** (of complex type **Address** with properties **Street**, **City** and **PostalCode**) and a navigation property **Orders**, unless stated otherwise.

Examples in this document routinely show the same payload twice, once in the format defined by [\[OData-JSON\]](#) and once in the format defined by this document, to make the correspondence between them apparent. Where an example shows only one of the two, the other is not relevant to the point being made.

All other text is normative unless otherwise labeled.

2 Compact JSON Format Design

[\[OData-JSON\]](#) extends JSON by defining general conventions for name/value pairs that annotate a JSON object, property or array, and a set of canonical name/value pairs for control information such as ids, types and links.

This format extends [\[OData-JSON\]](#) further by allowing a structured instance – an entity or a complex value – to be represented as a JSON array instead of a JSON object. Everything else defined by [\[OData-JSON\]](#) continues to apply.

2.1 Design Principles

The following principles govern this format. They are stated here because they explain, and constrain, the rules in the remainder of this document.

1. *Compact JSON is a superset of the OData JSON format.* Every payload that is valid according to [\[OData-JSON\]](#) is also a valid compact JSON payload. A receiver that accepts compact JSON therefore accepts strictly more payloads than a receiver that accepts only [\[OData-JSON\]](#), and a sender is never forced to use a positional representation where it is inconvenient or impossible.
2. *The saving is in the repetition.* A positional representation removes property names from a payload; the mapping from positions to properties is transmitted once. Consequently the benefit grows with the number of instances that share a positional property list, and is negligible or negative for a payload containing a single instance. This format therefore never *requires* a positional representation.
3. *The context URL describes the payload.* A receiver that has the context URL, and the metadata it references, can interpret a compact payload without knowledge of the request that produced it. This format does not introduce a second, competing mechanism for describing payload shape.
4. *Requests and responses are treated alike.* Large collections occur in request bodies – of **POST**, **PUT** and **PATCH** requests – as well as in response bodies, and the saving is the same in both directions. Where requests and responses must differ, the difference is called out explicitly.

Example 2: because of the [superset principle](#), a sender that cannot produce a positional representation for a particular instance may fall back to the representation defined by [\[OData-JSON\]](#) for that instance alone, within an otherwise positional payload:

```
{
  "@context": "$metadata#Customers(ID,Name)",
  "value": [
    ["ALFKI", "Alfreds Futterkiste"],
    { "ID": "ANATR", "Name": "Ana Trujillo Emparedados y helados" },
    ["ANTON", "Antonio Moreno Taquería"]
  ]
}
```

2.2 Relationship to the OData JSON Format

This document specifies only where the compact JSON format deviates from [\[OData-JSON\]](#). It is not a self-contained format specification: for the representation of primitive values, for the definition and meaning of control information, for URL and relative URL handling, and for everything else it does not mention, the rules of [\[OData-JSON\]](#) apply unchanged.

In particular, this format changes only *how a structured instance is laid out*. It does not change which information may appear in a payload, what that information means, or which requests a service supports.

3 Requesting the Compact JSON Format

The compact JSON format can be requested using the `$format` query option in the request URL with the media type `application/json` followed by the `compact=true` format parameter, optionally followed by other format parameters.

Alternatively, this format can be requested using the `Accept` header with the media type `application/json` followed by the `compact=true` format parameter, optionally followed by other format parameters.

If specified, `$format` overrides any value specified in the `Accept` header.

The names and values of the format parameters are case-insensitive.

A service that does not support the compact JSON format **MUST NOT** return a compact payload. Because the `compact` format parameter selects a payload shape that a receiver unaware of this specification will misinterpret, a service **MUST NOT** return a compact response unless the client requested it with `compact=true`, and **MUST** return **406 Not Acceptable** if the client requests `compact=true` and the service does not support it.

A client **MUST NOT** send a compact request body unless it has established that the service supports the compact JSON format, either from the [Capabilities.SupportedFormats](#) annotation or from out-of-band knowledge. A service that receives a request body labeled with `compact=true` that it does not support **MUST** reject the request with **415 Unsupported Media Type**.

3.1 Format Parameters

The format parameters defined in [\[OData-JSON\]](#) apply to the compact JSON format with the meaning defined there, subject to the following.

The `streaming` parameter is meaningful and **MAY** be specified. A positional representation is inherently ordered, and a compact payload that meets the [payload ordering constraints](#) **MUST** include `streaming=true`.

The `IEEE754Compatible` parameter is meaningful and **MUST** be included if `Edm.Int64` and `Edm.Decimal` numbers are represented as strings.

The `metadata` parameter is meaningful and **MAY** be specified with the value `minimal` or `none`. Note that `metadata=none` does not remove the [context](#) control information from a compact payload: the context URL determines the [positional property list](#) and is therefore required for the payload to be interpretable at all. See [section 7.4.1](#).

The value `metadata=full` is **NOT RECOMMENDED** with `compact=true`. Full metadata requires control information to be present for every instance, which forces every instance into a [wrapper object](#) and defeats the purpose of the format. Services **MAY** reject the combination with **406 Not Acceptable**.

3.2 Advertising Support

Services **SHOULD** advertise the supported media types by annotating the entity container with the term [Capabilities.SupportedFormats](#) defined in [\[OData-VocCap\]](#), listing all available formats and combinations of supported format parameters, including the `compact=true` format parameter if the compact JSON format is supported by the service.

Example 3: a service advertising support for both the format defined in [\[OData-JSON\]](#) and the compact JSON format

```
"@Capabilities.SupportedFormats": [
  "application/json;IEEE754Compatible=true",
```

```
"application/json;IEEE754Compatible=true;compact=true"  
]
```

4 Compact Representations

This section defines the two representations that this format adds to [\[OData-JSON\]](#): the *positional representation*, which conveys a structured instance as a JSON array, and the *wrapper object*, which conveys whatever needs a name in a place where a positional representation has no room for one. Everything else in this document is expressed in terms of these two.

4.1 Positional Representation

A structured instance – an entity or a complex value – MAY be represented as a JSON array instead of a JSON object. Such an array is called the *positional representation* of the instance.

The items of the array are the values of the instance's properties. A property name is not transmitted with the value; the property a value belongs to is identified by the position of the value within the array.

Example 4: the same entity in the format defined by [\[OData-JSON\]](#) and in its positional representation

```
{
  "@context": "$metadata#Customers(ID,Name,City)/$entity",
  "ID": "ALFKI",
  "Name": "Alfreds Futterkiste",
  "City": "Berlin"
}
```

```
{
  "@context": "$metadata#Customers(ID,Name,City)/$entity",
  "$": ["ALFKI", "Alfreds Futterkiste", "Berlin"]
}
```

A sender MAY choose the positional representation for some instances in a payload and the representation defined in [\[OData-JSON\]](#) for others; see the [superset principle](#). A receiver distinguishes the two by the JSON type of the instance: a JSON array is a positional representation, a JSON object is not.

4.2 Positional Property List

The *positional property list* of a structured instance is the ordered list of properties that its positional representation conveys.

If an instance is represented positionally:

- the number of items in the array MUST equal the number of items in the positional property list, and
- item *n* of the array MUST be the value of item *n* of the positional property list, formatted as described in [section 4.6](#).

A sender MUST NOT omit an item, MUST NOT add an item, and MUST NOT reorder items. A property whose value is null is represented by the JSON value `null` in its position; a property that has no value is represented as described in [section 4.6](#).

Note that this is a stricter requirement than the one [\[OData-JSON\]](#) places on a JSON object representation, where a sender may omit a property whose value it does not wish to transmit. In a positional representation there is no way to omit a value without shifting every subsequent value, so the positional property list must be transmitted in full. If a sender wishes to transmit fewer properties, it narrows the positional property list – by narrowing the select-list in the context URL – rather than shortening the array.

4.3 Determining the Positional Property List

The positional property list of an instance is determined by the select-list, as defined in [\[OData-Protocol, section 10\]](#), that applies to that instance.

The context URL of a compact payload MUST contain a select-list, and that select-list MUST enumerate every property conveyed positionally, at every level of nesting. In particular:

- the select-list MUST NOT be omitted, and MUST NOT be empty;
- the select-list MUST NOT contain the shortcut `*`, nor the shortcut `{namespace}.*` for the bound operations of a type;
- a select-item for a structured property whose value is conveyed positionally MUST carry a nested select-list, rather than the empty parentheses that [\[OData-Protocol\]](#) permits;
- the rule of [\[OData-Protocol\]](#) whereby a select-list containing only expanded navigation properties implicitly selects all structural properties does not apply to a compact payload.

This is a requirement on the *context URL*, not on the request. A client may use `$select=*`, or omit `$select` altogether, or use `$expand=*`; the service resolves the request as it normally would and then enumerates, in the context URL, the properties it has actually placed in the positional representation. A service may still convey further properties, such as dynamic properties of an open type, by name in a [wrapper object](#); see [section 6.8](#).

The reason for this requirement is that no other route to the positional property list is well defined. Deriving it from the CSDL document would require the receiver to know which version of that document the sender used, and to rely on the order in which properties are declared there, which [\[OData-CSDL\]](#) does not make significant. A service always knows which metadata it used; a client composing a request payload may not, and cannot determine it from the payload alone. Enumerating the select-list places the information with the party that reliably has it.

Let *T* be the type of the instance and *S* the sequence of select-list items, in the order in which they appear in the context URL, that applies to the instance. The positional property list is determined as follows:

1. Each item of *S* that begins with a type-cast segment — a qualified type name followed by a forward slash — is removed from *S* unless *T* is that type or is derived from it. From each such item that remains, the leading type-cast segment is removed.
2. The items of *S* are grouped as described in [section 4.4](#). Each group occupies exactly one position, at the position of the first of its items.
3. The positional property list is the resulting sequence of groups, in order.

Step 1 is what makes the positional property list depend on the instance and not only on the context URL: two instances of different types in one collection have different positional property lists. See [section 6.7](#).

Example 5: a select-list determines both the membership and the order of the positional property list; the two requests differ only in the order of the \$select items and produce different positional representations of the same entity

```
GET ~/Customers('ALFKI')?$select=Name,ID
```

```
{
  "@context": "$metadata#Customers(Name,ID)/$entity",
  "$": ["Alfreds Futterkiste", "ALFKI"]
}
```

```
GET ~/Customers('ALFKI')?$select=ID,Name
```

```
{
  "@context": "$metadata#Customers(ID,Name)/$entity",
```

```

"$": ["ALFKI", "Alfreds Futterkiste"]
}

```

Example 6: the request specifies no `$select`, so the service resolves it to the properties it chooses to return and enumerates those in the context URL. The client need not have asked for them by name; the context URL still says exactly what each position holds.

```
GET ~/Customers
```

```

{
  "@context": "$metadata#Customers(ID,Name,Address(Street,City,PostalCode))",
  "$": [
    ["ALFKI", "Alfreds Futterkiste", ["Obere Str. 57", "Berlin", "12209"]],
    ["ANATR", "Ana Trujillo", ["Avda. de la Constitución 2222", "México D.F.", "05021"]]
  ]
}

```

Example 7: a request that expands a navigation property without selecting anything. In the format defined by [\[OData-JSON\]](#) the context URL would be `$metadata#Customers(Orders(ID))`, leaving the structural properties implicit; a compact payload enumerates them.

```
GET ~/Customers?$expand=Orders($select=ID)
```

```

{
  "@context": "$metadata#Customers(ID,Name,Address(Street,City,PostalCode),Orders(ID))",
  "$": [
    [
      "ALFKI", "Alfreds Futterkiste", ["Obere Str. 57", "Berlin", "12209"],
      [[10643], [10692]]
    ]
  ]
}

```

4.4 Grouping of Select Items

A select-list may contain several items that address the same property of *T*: [\[OData-Protocol\]](#) represents a selected sub-property of a complex property using path syntax, so selecting two sub-properties of the same complex property yields two items sharing a first path segment. The positional representation gives such a property a single position.

The *first segment* of a select-item is the item with any `(...)` or `+ (...)` suffix removed, truncated before the first forward slash (/). Leading type-cast segments have already been removed by step 3 of [section 4.3](#) and so do not occur here.

Two items of *S* belong to the same group if and only if their first segments are equal. The group occupies the position of the first of its items.

If the property addressed by the first segment of a group is of a structured type, the *nested select-list* of that group is formed by concatenating, for each item of the group in order:

- for an item of the form `P/rest`, the item `rest`;
- for an item of the form `P(nested)` or `P+(nested)`, the items of `nested`;
- for an item of the form `P`, nothing.

The nested select-list MUST NOT be empty, for the reason given in [section 4.3](#): there is no well-defined default to fall back on.

The nested select-list determines the positional property list of the instances of that property, applying this section recursively.

Example 8: two selected sub-properties of the complex property `Address` share one position, which holds the positional representation of the complex value

```
GET ~/Customers?$select=Name,Address/City,Address/PostalCode
```

```
{
  "@context": "$metadata#Customers (Name,Address/City,Address/PostalCode) ",
  "$": [
    ["Alfreds Futterkiste", ["Berlin", "12209"]],
    ["Ana Trujillo", ["México D.F.", "05021"]]
  ]
}
```

The positional property list of each **Customer** is (**Name**, **Address**), and the positional property list of each **Address** is (**City**, **PostalCode**).

Example 9: a bare item and a suffixed item addressing the same navigation property form one group and therefore one position

```
$metadata#Employees/Sales.Manager (DirectReports,DirectReports+ (FirstName,LastName) )
```

The positional property list of each **Manager** is (**DirectReports**).

4.5 The Wrapper Object

A positional representation is a JSON array, and a JSON array has no name/value pairs. Anything that has to be conveyed *by name* alongside a value therefore has nowhere to go: the annotations and control information that [\[OData-JSON\]](#) attaches to the object representing an instance, or to the object containing a property; and any property that cannot be placed in the positional property list at all. The containing instance is no help, since it may itself be an array.

This format introduces a single construct for all of them, used uniformly wherever a value would otherwise appear.

A *wrapper object* is a JSON object whose name/value pairs are

- annotations and control information that apply to a value,
- optionally, that value itself, under the reserved name \$, and
- optionally, properties of the instance that are not in its positional property list, by name.

The name \$ is not a simple identifier ([\[OData-CSDL\]](#)) — a simple identifier is at least one character long and begins with an underscore or a Unicode letter — so it can never be the name of a declared or dynamic property, and [\[OData-JSON\]](#) never uses it. A receiver therefore distinguishes a wrapper object from the representation defined by [\[OData-JSON\]](#) as follows:

- a JSON object containing a name/value pair named \$ is a wrapper object;
- a JSON object all of whose name/value pairs are annotations or control information is a wrapper object that carries no value;
- any other JSON object is the representation defined by [\[OData-JSON\]](#).

The value under \$ is whatever representation applies to that value: the positional representation of a structured instance, a JSON object as defined in [\[OData-JSON\]](#), a JSON array for a collection, or a JSON primitive where the wrapper stands at the position of a primitive property. It is not restricted to arrays — a wrapper carrying an annotation on a primitive property holds that primitive under \$.

Properties carried by name in a wrapper object **MUST NOT** include a property that occupies a position in the instance's positional property list, and **MUST** follow the value, as required by [section 5.3](#). Carrying properties by name is what allows an instance of an open type to keep its positional representation while conveying dynamic properties that could not be placed in the select-list; see [section 6.8](#).

A wrapper object **MAY** appear wherever a value may appear:

- as the message body, in which case it carries the payload's [context](#) and the payload's content;

- at a position in a positional representation, in which case it carries the annotations of the property at that position and the property's value;
- as an item of a collection, in which case it carries the annotations of that member of the collection and the member itself.

A wrapper object that carries no value denotes a property or instance that has no value, as distinct from one whose value is null. The empty JSON object `{ }` carries neither annotations nor a value and therefore denotes “no value”; it is used at the position of a selected dynamic property that an instance does not have, see [section 6.8](#). Note that `{ }` satisfies the second bullet above vacuously rather than by construction: it is treated as a wrapper carrying no value by convention, being the only reading that is useful.

The name of the value in a wrapper object is `$`, in every position in which a wrapper object may appear.

Wherever [\[OData-JSON\]](#) specifies that the message body contains a name/value pair whose name is `value`, that name is also recognized as the name of the wrapper object's value and denotes the same thing. This follows from the [superset principle](#) and grants nothing beyond what [\[OData-JSON\]](#) already requires: the message bodies in which it applies are exactly those enumerated there, and this document does not add to them.

Everywhere else, `value` is not the name of a wrapper object's value. In particular, where [\[OData-JSON\]](#) represents the message body as the instance itself – for a single entity, a single complex value, or a single entity reference – a name/value pair named `value` in that message body is a *property* named `value`, and a receiver MUST NOT read it as the value of a wrapper object. A single entity or complex value represented positionally at the root of the message body therefore uses `$`.

Producers of compact payloads SHOULD use `$` wherever this document permits a choice.

This restriction is what keeps the two representations distinguishable. Were `value` also the wrapper's value name at the root of a message body representing a single entity, then `{"@context": "...#Customers/$entity", "value": [...] }` would be at once the positional representation of an entity and the [\[OData-JSON\]](#) representation of an entity having a collection-valued property named `value`, with nothing to tell the two apart.

Example 10: the same information three times — as defined by [\[OData-JSON\]](#), compact with `$` at every level, and compact with `value` at the root. The third form is permitted only because the message body is a collection, which is one of the cases in which [\[OData-JSON\]](#) itself uses `value`

```
{
  "@context": "$metadata#Customers(Name,Orders(ID))",
  "value": [
    {
      "Name": "Alfreds Futterkiste",
      "Orders@count": 2,
      "Orders": [{ "ID": 10643 }, { "ID": 10692 } ]
    }
  ]
}
```

```
{
  "@context": "$metadata#Customers(Name,Orders(ID))",
  "$": [
    ["Alfreds Futterkiste", { "@count": 2, "$": [[10643], [10692]] } ]
  ]
}
```

```
{
  "@context": "$metadata#Customers(Name,Orders(ID))",
  "value": [
    ["Alfreds Futterkiste", { "@count": 2, "$": [[10643], [10692]] } ]
  ]
}
```

```
  ]
}
```

4.6 Position Values

The value at a position is one of the following:

- the representation, as defined in [\[OData-JSON\]](#) or in this document, of the value of the property at that position in the positional property list;
- a [wrapper object](#), if annotations or control information apply to that value, or if the property has no value; or
- `null`, if the property is null.

The representation of a primitive value is unchanged from [\[OData-JSON\]](#).

A single-valued structured property holds either the positional representation of its value, a JSON object as defined in [\[OData-JSON\]](#), or `null`.

A collection-valued property holds a JSON array whose items are the representations of the members of the collection. An empty collection is represented as an empty JSON array.

Example 11: a collection-valued complex property; the outer array is the collection, each inner array the positional representation of one Address

```
{
  "@context": "$metadata#Customers(Name,Addresses/City)/$entity",
  "$": [
    "Alfreds Futterkiste",
    [ ["Berlin"], ["Hamburg"] ]
  ]
}
```

Note the consequence of the two preceding rules: for a collection-valued structured property, the value at the position is an array of arrays. A receiver that knows the positional property list also knows, from the metadata, whether a property is collection-valued, and can therefore distinguish the two nestings without ambiguity.

A property that has no value at all – as opposed to a property whose value is null – occurs when only annotations were requested for it, for example when a navigation property was expanded with `$count` only. Such a position holds a [wrapper object](#) carrying the annotations and no value. See [section 7.2](#).

5 Common Characteristics

5.1 Header Content-Type

Requests and responses with a compact JSON message body **MUST** have a **Content-Type** header value of **application/json** and **MUST** include the **compact** parameter with a value of **true**.

All other rules for the **Content-Type** header defined in [\[OData-JSON\]](#) apply unchanged, including the **charset**, **metadata**, **IEEE754Compatible** and **streaming** parameters.

*Example 12: a compact response that follows the payload ordering constraints and represents **Edm.Int64** and **Edm.Decimal** values as strings*

```
Content-Type: application/json;compact=true;metadata=minimal;
             IEEE754Compatible=true;streaming=true
```

5.2 Message Body

Each message body is represented as a single JSON object, as defined in [\[OData-JSON\]](#), with the single exception described in [section 8.2](#). A response message body is always a JSON object: the positional representation applies to the instances *within* the payload, not to the payload as a whole.

This object is either

- the representation of an [entity](#), an [entity reference](#) or a [complex value](#) that is not represented positionally, or
- a [wrapper object](#), whose value is the correct representation for the payload's content.

The name of the value in a wrapper object is **\$**. The name **value** is also recognized, but only in those message bodies in which [\[OData-JSON\]](#) itself uses it; see [section 4.5](#).

Example 13: a message body containing a collection of entities represented positionally

```
{
  "@context": "$metadata#Customers(ID,Name)",
  "$": [
    ["ALFKI", "Alfreds Futterkiste"],
    ["ANATR", "Ana Trujillo Emparedados y helados"]
  ]
}
```

Receivers **MUST** retain the order of items within an array in a compact payload. In a positional representation the order of items is significant and carries the mapping from values to properties; a receiver that reorders array items loses information.

5.3 Payload Ordering Constraints

The ordering constraints defined in [\[OData-JSON\]](#) apply to the JSON objects in a compact payload, and **MUST** only be assumed if explicitly indicated by the **streaming=true** format parameter in the **Content-Type** header.

A positional representation is inherently ordered, and satisfies by construction the constraints that [\[OData-JSON\]](#) imposes on the sequence of properties within an instance. The constraints that remain meaningful for a compact payload are those on the JSON objects it still contains, namely the message body and any [wrapper object](#):

- If present, the [context](#) control information **MUST** be the first name/value pair in the JSON object.
- For deleted entities, the **removed** control information **MUST** appear after **context**, if present, and before any other name/value pair.

- The **type** control information, if present, MUST appear next in the JSON object.
- The **id** and **etag** control information MUST appear before the value.
- The value, named **\$** or **value**, MUST appear after all annotations and control information, and before any properties carried by name as described in [section 4.5](#). The one exception is the **nextLink** of a collection, which MAY appear after the collection it annotates.

The requirement that **type** precede the value is load-bearing in a compact payload rather than merely conventional. The positional property list of an instance depends on the instance's type, so a receiver reading the payload as a stream must have the type before it reaches the positional representation. See [section 6.7](#).

Producers of compact payloads are encouraged to follow the payload ordering constraints whenever possible and to include the **streaming=true** format parameter.

6 Structured Instances

This section describes how each kind of OData value is represented in a compact payload. Only the deviations from [\[OData-JSON\]](#) are described; everything else is unchanged.

6.1 Entity

An entity MAY be represented as a JSON array, as described in [section 4.1](#). Its positional property list is determined from the select-list applying to it, as described in [section 4.3](#).

Control information that applies to the entity – **id**, **etag**, **type**, **editLink**, **readLink** and others – cannot appear inside the array. An entity to which control information applies is represented either as a [wrapper object](#) carrying that control information together with the positional representation, or as a JSON object as defined in [\[OData-JSON\]](#).

Example 14: a single entity with an ETag

```
{
  "@context": "$metadata#Customers(ID,Name)/$entity",
  "@etag": "W/\"MjAyNC0wMi0yOA==\"",
  "$": ["ALFKI", "Alfreds Futterkiste"]
}
```

6.2 Complex Value

A complex value MAY be represented as a JSON array, as described in [section 4.1](#). Its positional property list is determined from the nested select-list applying to it, as described in [section 4.4](#).

A null complex value is represented as **null**, not as an empty array.

6.3 Collection of Entities

A collection of entities is represented as a JSON array whose items are the representations of the entities in the collection, each of which MAY be a positional representation.

All entities in a collection share the same positional property list, which is determined once from the select-list applying to the collection.

Example 15: a collection of entities, showing the format defined by [\[OData-JSON\]](#) and the compact format side by side

```
GET ~/Customers?$select=ID,Name
```

```
{
  "@context": "$metadata#Customers(ID,Name)",
  "value": [
    { "ID": "ALFKI", "Name": "Alfreds Futterkiste" },
    { "ID": "ANATR", "Name": "Ana Trujillo" },
    { "ID": "ANTON", "Name": "Antonio Moreno" }
  ]
}
```

```
GET ~/Customers?$select=ID,Name
```

```
{
  "@context": "$metadata#Customers(ID,Name)",
  "$": [
    ["ALFKI", "Alfreds Futterkiste"],
    ["ANATR", "Ana Trujillo"],
    ["ANTON", "Antonio Moreno"]
  ]
}
```

```
  ]
}
```

6.4 Collection of Complex Values

A collection of complex values is represented as a JSON array whose items are the representations of the complex values in the collection, each of which MAY be a positional representation.

All complex values in a collection share the same positional property list.

6.5 Primitive Values

A primitive value is represented exactly as defined in [\[OData-JSON\]](#). A collection of primitive values is represented as a JSON array of such values.

Primitive values are already represented without a repeated property name and this format does not change them.

6.6 Navigation Properties

A navigation property that is expanded occupies a position in the positional property list of the containing entity, and the value at that position is the representation of the related entity or collection of related entities.

A navigation property that is not expanded and not selected does not occupy a position.

The **navigationLink** and **associationLink** control information for a navigation property, when present, is carried in a [wrapper object](#) at the navigation property's position, as described in [section 7.2](#).

*Example 16: an expanded collection-valued navigation property; the position of **Orders** holds the collection, whose items are the positional representations of the individual orders*

```
GET ~/Customers?$select=ID&$expand=Orders($select=ID,Amount)
```

```
{
  "@context": "$metadata#Customers(ID,Orders(ID,Amount))",
  "$": [
    ["ALFKI", [[10643, 29.46], [10692, 61.02]]],
    ["ANATR", []]
  ]
}
```

6.7 Derived Types

A collection may contain entities or complex values of types derived from the type of the collection.

The positional property list of an instance is determined from the select-list *and from the type of that instance*, as described in [section 4.3](#). Instances of different types within one collection therefore have different positional property lists: a select-item qualified with a type cast contributes a position to instances of that type, and to instances of types derived from it, and contributes nothing to any other instance.

An instance of a derived type conveys the properties selected for its own type in addition to those selected for the type of the collection. It carries neither a position nor a placeholder for a property selected for a peer type.

Because the positional property list depends on the instance's type, a receiver cannot decode a positional representation without knowing that type. Therefore:

- A service **MUST** include the **type** control information for any instance whose positional property list differs from the positional property list of the type declared by the context URL. This applies irrespective of the value of the **metadata** format parameter, for the same reason that the **context** control information is always required; see [section 7.4.1](#).

- The **type** control information is carried in the [wrapper object](#) around the positional representation and **MUST** precede it, as required by [section 5.3](#).

A receiver **MUST NOT** infer the type of an instance from the number of items in its positional representation: two types may yield positional property lists of equal length.

*Example 17: a heterogeneous collection in which no property of a derived type is selected. The **type** control information distinguishes the instances, but both positional property lists are (ID, Name).*

```
{
  "@context": "$metadata#Customers(ID,Name)",
  "$": [
    ["ALFKI", "Alfreds Futterkiste"],
    { "@type": "#Model.VipCustomer", "$": ["ANATR", "Ana Trujillo"] }
  ]
}
```

Example 18: PreferredContact and Since are declared by Model.VipCustomer only. The positional property list of a Customer is (ID); that of a Model.VipCustomer is (ID, PreferredContact, Since).

```
GET ~/Customers?$select=ID,Model.VipCustomer/PreferredContact,Model.VipCustomer/Since
```

```
{
  "@context":
    "$metadata#Customers(ID,Model.VipCustomer/PreferredContact,Model.VipCustomer/Since)",
  "$": [
    ["ALFKI"],
    { "@type": "#Model.VipCustomer", "$": ["ANATR", "email", 2019] }
  ]
}
```

Example 19: with two peer derived types selected, each instance carries only the properties of its own type, and nothing for the other

```
GET ~/Customers?$select=ID,Model.VipCustomer/PreferredContact,Model.WholesaleCustomer/Terms
```

```
{
  "@context":
    "$metadata#Customers(ID,Model.VipCustomer/PreferredContact,Model.WholesaleCustomer/Terms)",
  "$": [
    ["ALFKI"],
    { "@type": "#Model.VipCustomer", "$": ["ANATR", "email"] },
    { "@type": "#Model.WholesaleCustomer", "$": ["ANTON", "NET30"] }
  ]
}
```

6.8 Open Types and Dynamic Properties

A dynamic property of an open type occupies a position in the positional property list if, and only if, it is explicitly selected and therefore appears in the select-list of the context URL.

If a dynamic property is selected, the service **MUST** include a value at its position for every instance, using the empty JSON object { } for an instance that does not have that dynamic property.

This differs from the treatment of a property of a derived type, and the difference is deliberate. A type cast in the select-list says which instances a property applies to, so an instance to which it does not apply simply has a shorter positional property list. A selected dynamic property says nothing about which instances have it, so it is part of the positional property list of every instance, and { } conveys that this instance has no such property — as distinct from **null**, which conveys that it has one whose value is null.

An instance MAY in addition carry dynamic properties that are *not* in its positional property list, by name, in the [wrapper object](#) holding its positional representation, as described in [section 4.5](#). A property that occupies a position MUST NOT also be carried by name.

Example 20: Nickname is selected and therefore occupies a position; the first customer does not have it

```
GET ~/Customers?$select=ID,Nickname

{
  "@context": "$metadata#Customers(ID,Nickname)",
  "$": [
    ["ALFKI", {}],
    ["ANATR", "Ana"]
  ]
}
```

Example 21: Nickname and Score were not selected. The instance keeps its positional representation and carries them by name, so only the instance that has them pays for them.

```
{
  "@context": "$metadata#Customers(ID,Name)",
  "$": [
    ["ALFKI", "Alfreds Futterkiste"],
    { "$": ["ANATR", "Ana Trujillo"], "Nickname": "Ana", "Score": 42 }
  ]
}
```

6.9 Entity References

An entity reference is represented as defined in [\[OData-JSON\]](#): a JSON object containing the **id** control information. It has no positional representation, because it has no positional property list – the context URL **#\$ref** and **#Collection(\$ref)** carry no select-list.

A collection of entity references is a JSON array of such objects.

6.10 Media Entities and Stream Properties

The **mediaReadLink**, **mediaEditLink**, **mediaEtag** and **mediaContentType** control information of a media entity is carried in the [wrapper object](#) around the entity's positional representation.

A stream property occupies a position in the positional property list if it is selected. The value at that position is a [wrapper object](#) carrying the stream's control information, or the value of the stream as defined in [\[OData-JSON\]](#).

7 Annotations and Control Information

[OData-JSON] attaches annotations and control information to a value by adding name/value pairs to the JSON object that represents the value, or to the object that contains it:

- an annotation on an instance is a name/value pair `@ns.term` on the object representing the instance;
- an annotation on a property is a name/value pair `Property@ns.term` on the object *containing* the property;
- control information follows the same two patterns, with a reserved term name, for example `@id` and `Orders@count`.

In a compact payload both patterns are carried by the [wrapper object](#) defined in [section 4.5](#). This section describes where each annotation and each piece of control information appears, and how its name differs from the name **[OData-JSON]** gives it.

The *value* of an annotation, or of control information, is represented exactly as defined in **[OData-JSON]**. It is never conveyed positionally: a positional representation requires a select-list, as [section 4.3](#) requires, and no select-list applies to the value of an annotation. A complex-valued annotation is therefore a JSON object whose names are the properties declared by the term's type — including where one of those properties happens to be named `value`, which is a property name like any other and is not the reserved name of a wrapper object's value.

7.1 Instance Annotations

An annotation that applies to a structured instance is represented as a name/value pair `@ns.term` in the wrapper object around that instance's positional representation.

The name of an instance annotation is unchanged from **[OData-JSON]**.

Example 22: an instance annotation on one entity of a collection

```
{
  "@context": "$metadata#Customers(ID,Name)",
  "$": [
    ["ALFKI", "Alfreds Futterkiste"],
    {
      "@Core.Messages": [{ "code": "1", "message": "Stale", "severity": "info" }],
      "$": ["ANATR", "Ana Trujillo"]
    }
  ]
}
```

7.2 Property Annotations

An annotation that applies to a property is represented as a name/value pair in the wrapper object at that property's position.

The name of a property annotation in a compact payload is `@ns.term`. The property name prefix that **[OData-JSON]** requires – as in `Property@ns.term` – is omitted, because the position already identifies the property.

Example 23: the `Core.ValueException` annotation of a property, in both formats. Note that the annotation's own value is unchanged between them: `value` there is a property declared by the term's type, not the name of a wrapper object's value, and it is not renamed to `$`. Note also that the `$` of the wrapper holds a number here, because the property it annotates is a primitive one.

```
{
  "@context": "$metadata#Customers(ID,Revenue)/$entity",
  "ID": "ALFKI",
  "Revenue@Core.ValueException": { "value": "1234567890123456789" },
}
```

```

    "Revenue": 1234567890123456800
  }

  {
    "@context": "$metadata#Customers(ID,Revenue)/$entity",
    "$": [
      "ALFKI",
      {
        "@Core.ValueException": { "value": "1234567890123456789" },
        "$": 1234567890123456800
      }
    ]
  }
}

```

Example 24: a collection-valued property for which only the count was requested; the position holds a wrapper object with an annotation and no value

```
GET ~/Customers?$select=Name,Addresses/$count
```

```

{
  "@context": "$metadata#Customers(Name,Addresses/$count)",
  "$": [
    ["Alfreds Futterkiste", { "@count": 2 }],
    ["Ana Trujillo", { "@count": 0 }]
  ]
}

```

Note that under [section 4.4](#) the select-item **Addresses/\$count** forms a group whose first segment is **Addresses**, so **Addresses** occupies a single position, and the count segment is what that position conveys. See also open issue 13.

7.3 Selected Annotations

An instance annotation that is explicitly selected appears in the select-list of the context URL and therefore occupies a position in the positional property list, like any other select-item. The value at that position is the value of the annotation.

This is the case where an annotation is requested as data. It is distinct from an annotation that merely accompanies a value, which is carried in a wrapper object as described in the preceding sections.

*Example 25: the annotation **@Model.Rating** is selected and occupies the second position*

```
GET ~/Customers?$select=Name,@Model.Rating
```

```

{
  "@context": "$metadata#Customers(Name,@Model.Rating)",
  "$": [
    ["Alfreds Futterkiste", 5],
    ["Ana Trujillo", 3]
  ]
}

```

Annotations requested through the **include-annotations** preference do not appear in the context URL, do not affect the positional property list, and are therefore carried in wrapper objects.

7.4 Control Information

Control information is represented in a compact payload in the same way as annotations, following the rules of the preceding sections. The following subsections describe the control information whose treatment in a compact payload warrants specific mention.

7.4.1 Control Information: `context`

The `context` control information is defined in [\[OData-JSON\]](#) and its value is the context URL of the payload, as defined in [\[OData-Protocol, section 10\]](#).

In a compact payload the context URL additionally determines the [positional property list](#) of the instances in the payload, as described in [section 4.3](#). It is therefore required for a compact payload to be interpretable.

A compact payload MUST include the `context` control information in the message body, and MUST include it in any nested wrapper object for which [\[OData-JSON\]](#) requires a context URL, irrespective of the value of the `metadata` format parameter. In particular, a payload labeled `metadata=none` MUST still include the `context` control information.

7.4.2 Control Information: `count` and `nextLink`

The `count` and `nextLink` control information of the collection in the message body is carried in the message body object, as in [\[OData-JSON\]](#).

For a nested collection, both are carried in the wrapper object at the collection's position.

Example 26: `count` and `nextLink` for the collection in the message body and for a nested collection

```
{
  "@context": "$metadata#Customers(Name,Orders(ID))",
  "@count": 137,
  "$": [
    [
      "Alfreds Futterkiste",
      {
        "@count": 42,
        "@nextLink": "Customers('ALFKI')/Orders?$skiptoken=10",
        "$": [[10643], [10692]]
      }
    ]
  ],
  "@nextLink": "Customers?$skiptoken=10"
}
```

7.4.3 Control Information: `type`

The `type` control information is carried in the [wrapper object](#) around the instance it applies to and MUST precede that instance's positional representation.

In a compact payload `type` is not merely informative. The positional property list of an instance depends on the instance's type, so a receiver cannot decode a positional representation without it. A service MUST include `type` for any instance whose positional property list differs from that of the type declared by the context URL, irrespective of the value of the `metadata` format parameter. See [section 6.7](#).

A receiver MUST NOT infer the type of an instance from the number of items in its positional representation.

8 Request Payloads

Large collections of entities and complex values occur in request bodies as well as in response bodies, and the saving from a positional representation is the same in both directions. A request body MAY therefore use the positional representation wherever a response body may.

A client MUST NOT send a compact request body unless it has established that the service supports the compact JSON format; see [section 3](#).

8.1 Determining the Positional Property List in Requests

[\[OData-Protocol\]](#) notes that request payloads generally do not require context URLs, because the type of the payload can be determined from the request URL. A compact request body needs more than the type: it needs the positional property list.

A compact request body MUST contain the [context](#) control information, and its context URL MUST carry a select-list meeting the requirements of [section 4.3](#). The positional property list is then determined from that select-list exactly as it is for a response.

This requirement is more pointed for requests than for responses. A service producing a response knows which version of the CSDL document it used; a client composing a request body may not know, and nothing in the request conveys it. Without an enumerated select-list there is no well-defined way for the service to learn what the client placed at each position.

The context URL of a request body MUST be valid for the resource addressed by the request URL, as defined in [\[OData-Protocol, section 10\]](#). It describes the body; it does not modify the request.

A service MUST reject with **400 Bad Request** a compact request body that omits the **context** control information, or whose context URL does not carry a conforming select-list.

A service MUST reject with **400 Bad Request** a compact request body whose positional representation does not have the number of items required by the positional property list.

8.2 Message Body of a Request

The message body of a compact request is a single JSON object, as it is for a response and as [\[OData-JSON\]](#) requires. It is never a bare JSON array.

[\[OData-JSON\]](#) wraps a request body in an object deliberately, so that there is always somewhere to put control information and annotations, and so that the shape of the body does not depend on whether any such information happens to be present. A compact request body has the further need to carry its [context](#), which a bare array could not do.

8.3 Creating an Entity

An entity to be created MAY be represented positionally. Properties that the client does not wish to specify – leaving them to the service to default – MUST NOT be represented by **null**, which would request the value null. They are instead excluded from the positional property list, which requires the request body to carry a **context** control information whose select-list enumerates exactly the properties being specified.

Example 27: creating an entity specifying only two properties, leaving the remainder to the service

```
POST ~/Customers
Content-Type: application/json;compact=true
```

```
{
  "@context": "$metadata#Customers(ID,Name)/$entity",
  "$": ["ALFKI", "Alfreds Futterkiste"]
}
```

8.4 Updating an Entity

The semantics of **PATCH** defined in [\[OData-Protocol\]](#) rest on the distinction between a property that is present in the request body, whose value is applied, and a property that is absent, which is left unchanged. In a positional representation a value cannot be absent without shifting every subsequent value.

The select-list of the **context** control information carries this distinction: the positional property list *is* the list of properties to be updated. A property that is not in the positional property list is absent from the request in the sense of [\[OData-Protocol\]](#) and is left unchanged. A property that is in the positional property list with the value **null** is set to null.

A **PATCH** request body that does not contain the **context** control information therefore specifies a value for every structural property of the type, which is rarely the intent. Clients **SHOULD** include the **context** control information in compact **PATCH** request bodies.

*Example 28: updating two properties of an entity and leaving all others unchanged; **Region** is set to null, **Name** is set to a new value, and every property not named in the context URL is unaffected*

```
PATCH ~/Customers('ALFKI')
Content-Type: application/json;compact=true
```

```
{
  "@context": "$metadata#Customers(Name,Region)/$entity",
  "$": ["Alfred's Futterkiste", null]
}
```

For **PUT**, the semantics defined in [\[OData-Protocol\]](#) apply unchanged: properties not specified in the request body are reset to their default values. As for **PATCH**, the properties specified are those in the positional property list.

8.5 Updating a Collection of Entities

A request body that updates a collection of entities is a [delta payload](#) with the context URL fragment **#\$delta**, as defined in [\[OData-Protocol, section 10.17\]](#). Its items **MAY** be represented positionally, as described in [section 9](#).

8.6 Deep Insert

A deep insert specifies related entities alongside the entity being created. Related entities occupy a position in the positional property list of the containing entity, exactly as an expanded navigation property does in a response, and the select-list of the **context** control information enumerates them using the same syntax.

Example 29: creating a customer together with two orders

```
POST ~/Customers
Content-Type: application/json;compact=true
```

```
{
  "@context": "$metadata#Customers(ID,Name,Orders(ID,Amount))/$entity",
  "$": [
    "ALFKI",
    "Alfreds Futterkiste",
    [[10643, 29.46], [10692, 61.02]]
  ]
}
```

```

    ]
  }

```

8.7 Bind Operations

A bind operation relates a new or updated entity to an existing entity. [\[OData-JSON\]](#) represents it as a name/value pair **Property@bind** or **Property@id** on the containing object.

In a compact request body the navigation property occupies a position in the positional property list, and the bind operation is carried in a [wrapper object](#) at that position, as a property annotation, following [section 7.2](#).

Example 30: creating an order bound to an existing customer

```

POST ~/Orders
Content-Type: application/json;compact=true

```

```

{
  "@context": "$metadata#Orders(ID,Amount,Customer)/$entity",
  "$": [
    10643,
    29.46,
    { "@id": "Customers('ALFKI')" }
  ]
}

```

8.8 Action and Function Parameters

The parameters of an action invoked with **POST**, or of a function invoked with parameter aliases, are represented as defined in [\[OData-JSON\]](#): a JSON object whose name/value pairs are the parameter names and values.

A parameter value **MUST NOT** use the positional representation. A parameter payload carries no context URL, and [\[OData-Protocol\]](#) defines no context URL template for one, so there is no way to convey the select-list that [section 4.3](#) requires.

This is a real limitation rather than a considered exclusion: an action taking a large collection of entities or complex values is exactly the kind of payload this format exists to compact. That no context URL template has been needed for a parameter payload so far — the case arises only in requests — is not a reason not to define one. The Technical Committee intends to develop proposals; see open issue 1(b).

Example 31: an action taking a collection of complex values. Each address is represented as defined in [\[OData-JSON\]](#), because no select-list is available for the parameter's declared type.

```

POST ~/Customers('ALFKI')/Model.AddAddresses
Content-Type: application/json;compact=true

```

```

{
  "addresses": [
    { "Street": "Obere Str. 57", "City": "Berlin", "PostalCode": "12209" },
    { "Street": "Neue Str. 1", "City": "Hamburg", "PostalCode": "20095" }
  ]
}

```

9 Delta Payload

A delta payload is a collection whose items are added or changed entities, deleted entities, added links and deleted links, as defined in [\[OData-JSON\]](#) and [\[OData-Protocol, section 10.17\]](#).

Delta payloads are among the payloads for which this format is most useful: a delta response repeats the same shape of entity many times, and, for a collection under active change tracking, may do so repeatedly over time.

All items of a delta payload share one context URL, whose fragment ends in `/$delta`, and therefore one positional property list, determined as described in [section 4.3](#) from the select-list preceding `/$delta`.

9.1 Added or Changed Entity

An added or changed entity MAY be represented positionally. Its positional property list is the one determined for the delta payload.

As for a **PATCH** request body, the positional property list is the list of properties conveyed: a property that is not in the positional property list is unchanged, and a property in the list whose value is `null` has been set to null.

The `id` control information, where required, is carried in a [wrapper object](#) around the positional representation.

Example 32: a delta payload containing one changed entity

```
GET ~/Customers?$deltatoken=1234
```

```
{
  "@context": "$metadata#Customers(ID,Name)/$delta",
  "$": [
    ["ALFKI", "Alfred's Futterkiste"]
  ],
  "@deltaLink": "Customers?$deltatoken=5678"
}
```

9.2 Deleted Entity

A deleted entity is identified by the **removed** control information and therefore requires a JSON object. It is represented as a [wrapper object](#) carrying **removed**, **id** and, where required, **context**.

A deleted entity in a compact payload carries no positional representation, because a deleted entity conveys no property values other than the key, which is conveyed by **id**.

Example 33: a delta payload containing one changed and one deleted entity

```
{
  "@context": "$metadata#Customers(ID,Name)/$delta",
  "$": [
    ["ALFKI", "Alfred's Futterkiste"],
    { "@removed": { "reason": "deleted" }, "@id": "Customers('ANATR')" }
  ],
  "@deltaLink": "Customers?$deltatoken=5678"
}
```

Note that this is exactly the representation defined by [\[OData-JSON\]](#) for a deleted entity, as the [superset principle](#) permits.

9.3 Added and Deleted Links

Added and deleted links are identified by the context URL fragments `/$link` and `/$deletedLink` and are represented as JSON objects containing **context**, **source**, **relationship** and **target**, exactly as defined in [\[OData-JSON\]](#).

They have no positional representation. A link carries three values whose names are fixed by this specification rather than by the model, they do not appear in a select-list, and there is consequently no positional property list for them.

9.4 Nested Delta Payloads

A nested delta payload, representing changes to a related collection, occupies the position of the navigation property in the positional property list of the containing entity, and is itself a collection whose items follow the rules of this section.

10 Service Document

The service document is represented exactly as defined in [\[OData-JSON\]](#).

The entries of a service document have no positional property list: they are not instances of a type declared in the model, their members are fixed by [\[OData-JSON\]](#) rather than by the model, and a service document is small and requested once. A compact representation would carry no benefit.

A service MUST accept a request for the service document with the `compact=true` format parameter and MUST respond with the representation defined in [\[OData-JSON\]](#), labeled `compact=true`. This follows from the [superset principle](#) and replaces the behavior of returning an error.

11 Error Response

An error response is represented exactly as defined in [\[OData-JSON\]](#).

The members of an error object are fixed by [\[OData-JSON\]](#) rather than by the model, no context URL and therefore no select-list applies to them, and an error response is not a large payload. A compact representation would carry no benefit.

The **details** member of an error remains a JSON array of JSON objects.

Example 34: an error returned in response to a request that specified `compact=true`

```
Content-Type: application/json;compact=true
```

```
{
  "error": {
    "code": "501",
    "message": "Unsupported functionality"
  }
}
```


12 Batch Requests and Responses

The JSON batch format defined in [\[OData-JSON\]](#) applies unchanged. The **requests** and **responses** members remain JSON arrays of JSON objects whose members are fixed by [\[OData-JSON\]](#).

The **body** of an individual request or response within a batch MAY use the compact JSON format, in which case the individual request or response MUST carry a **Content-Type** header including the **compact=true** format parameter, and the rules of this document apply to that body.

The **compact** format parameter of the batch request itself applies to the batch document, not to the bodies of the individual requests it contains. Because the batch document has no compact representation, a batch request labeled **compact=true** is identical to one that is not.

Example 35: a batch request in which one individual request carries a compact body

```
{
  "requests": [
    {
      "id": "1",
      "method": "post",
      "url": "Customers",
      "headers": { "content-type": "application/json;compact=true" },
      "body": {
        "@context": "$metadata#Customers(ID,Name)/$entity",
        "$": ["ALFKI", "Alfreds Futterkiste"]
      }
    }
  ]
}
```

13 Conformance

Conformance to this specification presupposes conformance to [\[OData-JSON\]](#). The requirements below are additional to, and do not replace, the conformance requirements of that specification.

In order to be a conforming consumer of the OData compact JSON format, a client or service:

1. MUST be a conforming consumer of the OData JSON format ([\[OData-JSON\]](#))
2. MUST accept, wherever a structured instance may appear, either a JSON object or a positional representation ([section 4.1](#))
3. MUST determine the positional property list from the context URL and the referenced metadata ([section 4.3](#))
 1. MUST reject a compact payload whose context URL carries no select-list, an empty select-list, or the shortcut *
 2. MUST group select-items that share a first segment into a single position ([section 4.4](#))
 3. MUST determine the positional property list of an instance of a derived type from that type, retaining only those type-cast select-items that apply to it ([section 6.7](#))
4. MUST NOT infer the positional property list from the number of items in a positional representation
5. MUST accept a [wrapper object](#) wherever a value may appear ([section 4.5](#))
 1. MUST accept the value name \$
 2. MUST accept the value name `value` in those message bodies in which [\[OData-JSON\]](#) uses it, and MUST NOT read `value` as a wrapper object's value name elsewhere
 3. MUST accept a wrapper object that carries no value
 4. MUST accept a wrapper object that carries properties by name after the value
6. MUST accept property annotations without the property name prefix ([section 7.2](#))
7. MUST accept the `context` control information in a payload labeled `metadata=none` ([section 7.4.1](#))
8. MUST be prepared to receive a payload in which some instances are represented positionally and others are not ([section 2.1](#))
9. MUST be prepared to receive the service document, error responses and batch documents in the representation defined by [\[OData-JSON\]](#) ([section 10](#), [section 11](#), [section 12](#))

In order to be a conforming producer of the OData compact JSON format, a client or service:

10. MUST be a conforming producer of the OData JSON format ([\[OData-JSON\]](#))
11. MUST NOT produce a compact payload unless the receiver has indicated that it accepts one ([section 3](#))
 1. a service MUST NOT return a compact response unless the request specified `compact=true`
 2. a client MUST NOT send a compact request body unless the service has advertised support for `compact=true`
12. MUST include the `compact=true` format parameter in the `Content-Type` header of every compact payload ([section 5.1](#))
13. MUST include the `context` control information in every compact message body, requests included, and MUST enumerate in its select-list every property conveyed positionally, at every level of nesting ([section 4.3](#))
 1. MUST include the `type` control information for any instance whose positional property list differs from that of the type declared by the context URL, irrespective of the `metadata` format parameter ([section 6.7](#))
14. MUST produce, for every positional representation, exactly as many items as the positional property list has entries, in that order ([section 4.2](#))
15. MUST carry a dynamic property that is not in the instance's positional property list by name in the wrapper object holding the positional representation, and MUST NOT carry any property both positionally and by name ([section 6.8](#))
16. MUST use the empty JSON object { } at the position of a selected dynamic property that the instance does not have ([section 6.8](#))

17. MUST NOT use **value** as the name of a wrapper object's value except in those message bodies in which [\[OData-JSON\]](#) uses it, and SHOULD use \$ throughout ([section 4.5](#))
18. MUST NOT use the shortcuts * or {**namespace**} . * in the context URL of a compact payload, and MUST NOT rely on the implicit selection of all structural properties ([section 4.3](#))

In order to be a conforming service supporting the OData compact JSON format, a service:

19. SHOULD advertise support with the [Capabilities.SupportedFormats](#) term ([section 3.2](#))
20. MUST return **406 Not Acceptable** if the client requests **compact=true** and the service does not support it ([section 3](#))
21. MUST reject with **415 Unsupported Media Type** a request body labeled **compact=true** that it does not support ([section 3](#))
22. MUST reject with **400 Bad Request** a compact request body whose positional representation does not match the positional property list, or which omits the **context** control information or a conforming select-list ([section 8.1](#))

14 Open Issues

This section is non-normative. It records the decisions that the drafting of this document forced, but that the Technical Committee has not yet taken. Each item states the position currently taken in the draft, so that the draft is internally consistent, together with the alternatives considered.

14.1 Cross-Specification Dependencies

1. **Context URLs in request payloads.** *Merged with former issue 14; to be put to the TC as a single proposal.* Two request payloads need a context URL that [\[OData-Protocol\]](#) does not currently provide.

(a) *Request bodies.* [\[OData-Protocol, section 10\]](#) states that request payloads generally do not require context URLs, and defines a template for a request body only for the delta payload used to update a collection (`#$delta`). Since D-29 a compact request body **MUST** carry one, both to convey the positional property list and, for a **PATCH**, to convey which properties are affected. Without a Part 1 change, compact request bodies cannot be specified at all — this is a blocker for chapter 8, not a refinement. *Position taken:* the draft uses response context URL templates in request bodies unchanged. *Alternative:* define request-specific templates in Part 1, or define separate control information for the positional property list, which would contradict design principle 3 in [section 2.1](#).

(b) *Action and function parameter payloads.* A parameter payload has no context URL and no template for one, so [section 8.8](#) currently forbids the positional representation there (D-30). This is a real loss: an action taking a large collection of entities or complex values is exactly what this format exists to compact. That no context URL has been needed for a parameter payload until now — the case arises only in requests — is no reason not to define one. *Candidate directions, none yet assessed:* a template addressing the operation and carrying a select-list per parameter; a select-list carried per parameter within the parameter object; or deriving the list from the parameter's declared type together with an explicit metadata document version.

2. **Losslessness.** The stated goal is that every construct of [\[OData-JSON\]](#) has a compact counterpart, and that every [\[OData-JSON\]](#) payload is a valid compact payload. The second half holds by the [superset principle](#). The first half has not been verified exhaustively against [\[OData-JSON\]](#); the constructs currently *without* a positional counterpart are entity references, links in delta payloads, the service document, errors and the batch document, each for the reason given in the respective section. The Technical Committee should decide whether that list is acceptable, and whether [\[OData-JSON\]](#) should change where it is not.
3. **Stability of property order.** *Resolved.* The draft no longer derives the positional property list from the CSDL document at all: [section 4.3](#) requires the context URL of every compact payload to enumerate the select-list, forbids the shortcuts `*` and `{namespace}.*`, and disapplies the implicit selection of all structural properties. The dependency on declaration order, and with it the need for versioned metadata document URLs, is gone.

The decisive argument was not that declaration order is unstable but that it is *unknowable to the reader*: a service always knows which version of the CSDL document it used, whereas a client composing a request payload does not, and cannot determine it from the payload. Enumerating the select-list places the information with the party that reliably has it.

Note that this constrains the context URL, not the request: `$select=*`, `$expand=*` and an absent `$select` all remain valid, and the service enumerates what it actually returned.

14.2 Naming and Encoding Decisions

4. **The value name.** *Decided:* `$`. The name of a wrapper object's value is `$.`, used in earlier drafts, is a valid simple identifier ([\[OData-CSDL\]](#)) and therefore collides: a type declaring, or an open type carrying, a property named `_` could not be told apart from a wrapper object. A simple identifier is at least one character long and begins with an underscore or a Unicode letter, so `$`, `@` and the empty string are the collision-free candidates; the TC chose `$`.

The reasons, recorded because the choice determines how a receiver recognizes a wrapper object at all:

- [\[OData-CSDL\]](#) JSON already solves this exact problem the same way — “to avoid name collisions, all fixed member names are prefixed with a dollar (\$) sign” — and it does so in objects that mix all three kinds of member the compact format now needs in one place: \$-prefixed fixed names, @-prefixed annotations, and model-defined names. Adopting \$ gives the two formats the same three-way split.
- @ was considered and rejected. Because [\[OData-JSON\]](#) parsers conventionally separate annotations from properties by testing whether a name begins with @, a value named @ would fall into the annotation branch of existing code, silently.
- The empty string was considered and rejected. It is legal JSON and collision-free, but "" and " " are indistinguishable in fonts, diffs and pasted payloads, so a stray space yields valid JSON with the wrong meaning and no diagnostic; it cannot be named in prose or found by eye in an example; and it is the least-tested path in JSON tooling. Its only advantage is one byte per wrapper object, and wrapper objects occur only where annotations apply, so compactness does not decide the question.

Separately, `value` is recognized as the name of the wrapper object’s value only in those message bodies in which [\[OData-JSON\]](#) itself uses it — a single primitive value and the collections enumerated there — and not where [\[OData-JSON\]](#) represents the message body as the instance itself. An earlier draft recognized `value` at every root; that was withdrawn on review because it both extended [\[OData-JSON\]](#) and made a single-entity message body ambiguous. See [section 4.5](#).

5. **“Not applicable” at a position.** *Resolved, narrowed in scope.* The empty JSON object `{ }` denotes “no value” at the position of a selected dynamic property that an instance does not have ([section 6.8](#)). It is no longer used for properties of derived types, which under issue 6 simply do not occupy a position on instances of other types. *Accepted consequence:* now that a wrapper object is recognized by the presence of \$, `{ }` is not a wrapper by that test. The TC accepted that `{ }` conveys “undefined” by convention rather than as a consequence of the definition, and [section 4.5](#) says so. *Alternative not taken:* `null`, which is shorter by two bytes but conflates “no such property” with “property is null” and so is not lossless.
6. **Wrapper objects carrying named properties.** *Resolved and drafted.* A wrapper object may carry, after the value, properties of the instance that are not in its positional property list. An instance of an open type therefore keeps its positional representation and conveys its unselected dynamic properties by name, instead of falling back wholesale to the representation defined by [\[OData-JSON\]](#). Only the instances that have such properties pay for them.

The recognition rule in [section 4.5](#) is consequently two-part: an object containing \$ is a wrapper; an object all of whose members are annotations is a wrapper carrying no value; anything else is the [\[OData-JSON\]](#) representation. This is what makes the choice of \$ load-bearing rather than cosmetic.

Still open: a property that occupies a position MUST NOT also be carried by name, but the draft gives a receiver no obligation to detect a producer that breaks this. Whether that should be an error, and which wins if it happens, is not settled.

7. **The compact format parameter.** The draft uses `compact=true`. Should the parameter instead take a value naming the compaction strategy, for example `compact=rows`, to leave room for a future column-oriented or dictionary-encoded variant without a new media type?

[14.3 Constructs Not Yet Specified](#)

8. **Recursive expansion.** [\[OData-Protocol\]](#) represents an expansion with `$levels` using the + infix, as in `DirectReports+(FirstName, LastName)`. [Section 4.4](#) folds such an item into the group of its navigation property but does not say how the nested select-list applies at the second and subsequent levels, where it may differ from the first.

9. **Operations in the select-list.** A select-list may contain actions and functions, and the shortcut `{namespace}.*`. Such an item occupies a position under the rules of [section 4.3](#), and the value at that position would be the operation advertisement, or the empty wrapper object if the operation is not available. This is a consequence of the rules rather than a decision, and has not been examined.
10. **Action and function parameters.** [Section 8.8](#) keeps the parameter object as defined in [\[OData-JSON\]](#) and allows only the parameter values to be positional. Parameters have a declared order in [\[OData-CSDL\]](#), so the parameter object could itself be a positional array. The saving is small, and the interaction with optional parameters and with parameter aliases has not been examined.
11. **Interaction with \$apply.** Aggregated results introduce dynamic properties that appear in the context URL, which suggests the rules of this document apply unchanged, but this has not been verified against [\[OData-Aggregation\]](#). Transformations that produce a heterogeneous result, such as `concat`, need particular attention.
12. **Consistency within a collection.** The draft allows a sender to mix positional and object representations freely within one collection. A receiver optimized for the positional case may prefer a guarantee that a collection is homogeneous, at the cost of forcing an entire collection into the object representation whenever a single instance requires it.
13. **The count segment in a select-list.** [\[OData-URL\]](#) allows `$select=ID,Addresses/$count` and `$expand=Orders/$count`, but neither [\[OData-Protocol\]](#) nor [\[OData-JSON\]](#) states explicitly how the count segment appears in the context URL. [Example 24](#) assumes it is retained as written, which makes the grouping rule of [section 4.4](#) produce the intended single position. If it is not retained, the positional property list cannot distinguish “count only” from “the collection itself” and this document needs a rule of its own.
14. **Positional parameter values for actions and functions.** *Merged into issue 1(b).*
15. **Control information in the positional representation.** Selected instance annotations already occupy positions; the proposal is to allow control information — `@id`, `@etag`, `@type` — to do so as well. This would repair the inversion recorded under D-20, whereby `metadata=full` currently forces every instance into a wrapper object and so defeats the format.

The select-list here is written by the service, not by the client. The context URL describes what the payload contains; `$select` is what the client asked for. D-29 already relies on that distinction — a service resolves `$select=*`, or no `$select` at all, and then enumerates what it actually returned. Whether `@id` and `@etag` appear is driven by the protocol, the model and the service's own choice, exactly as it is for the format defined in [\[OData-JSON\]](#) today; a client is not surprised to receive them. Extending `$select` to admit them would therefore be the wrong move. Only the context URL need admit them.

That materially narrows the cross-specification ask. An earlier reading of this issue held that both Part 1 and Part 2 would have to change, and that the change was larger than issue 1. On the reading above, `$select` is untouched and only the context URL grammar is affected.

Settled in principle:

- If type information is conveyed positionally, the `type` control information **MUST** occupy the **first** position (D-31). It is the only thing that must be read before the positional property list is known, so it is the only one whose position must be fixed.
- The service chooses, per payload, whether to convey a given piece of control information in the wrapper object or positionally. Neither is mandated.
- An instance lacking a given piece of control information uses `{ }` at that position, exactly as for a selected dynamic property (D-27).

Still to settle:

- **Which control information is eligible.** `count` and `nextLink` apply to a *collection*, not to an instance, so they have no per-instance position and are out of scope. The eligible set is the instance-scoped control information: `id`, `etag`, `type`, `editLink`, `readLink` and the media ones. An earlier draft of this issue wrongly cited `count`.
- **Whether the ABNF can separate the two select-lists.** The context URL grammar is defined by the [\[OData-ABNF\]](#) rule [\[context\]](#). If its select-list shares the [\[selectItem\]](#) rule with `$select`, then admitting control information in one and not the other needs a distinct rule rather than a widened shared one.
- **No duplication.** If the select-list places `type` at position 0, a wrapper object around that instance **MUST** NOT also carry `type`.
- **The base-type cost.** If `type` occupies position 0 then *every* instance carries something there, including instances of the declared type, which would use `{ }`. That trades a small fixed cost on every instance against removing the wrapper object from every derived instance. Which wins depends on how derived-heavy the collection is, so the choice belongs to the service — but the draft should say so, rather than leaving producers to guess.

Appendix A. References

This appendix contains the normative and informative references that are used in this document.

While any hyperlinks included in this appendix were valid at the time of publication, OASIS cannot guarantee their long-term validity.

A.1 Normative References

The following documents are referenced in such a way that some or all of their content constitutes requirements of this document.

[OData-ABNF]

ABNF components: OData ABNF Construction Rules Version 4.02 and OData ABNF Test Cases.

See link in [“Related work”](#) section on cover page.

[OData-CSDL]

OData Common Schema Definition Language (CSDL) JSON Representation Version 4.02.

See link in [“Related work”](#) section on cover page.

OData Common Schema Definition Language (CSDL) XML Representation Version 4.02.

See link in [“Related work”](#) section on cover page.

[OData-JSON]

OData JSON Format Version 4.02.

See link in [“Related work”](#) section on cover page.

[OData-Protocol]

OData Version 4.02. Part 1: Protocol.

See link in [“Related work”](#) section on cover page.

[OData-URL]

OData Version 4.02. Part 2: URL Conventions.

See link in [“Related work”](#) section on cover page.

[OData-VocCap]

OData Vocabularies Version 4.0: Capabilities Vocabulary.

See link in [“Related work”](#) section on cover page.

[OData-VocCore]

OData Vocabularies Version 4.0: Core Vocabulary.

See link in [“Related work”](#) section on cover page.

[RFC2119]

Bradner, S., “Key words for use in RFCs to Indicate Requirement Levels”, BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997. <https://www.rfc-editor.org/info/rfc2119>.

[RFC3986]

Berners-Lee, T., Fielding, R., and L. Masinter, “Uniform Resource Identifier (URI): Generic Syntax”, STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005. <https://www.rfc-editor.org/info/rfc3986>.

[RFC3987]

Duerst, M. and M. Suignard, “Internationalized Resource Identifiers (IRIs)”, RFC 3987, DOI 10.17487/RFC3987, January 2005. <https://www.rfc-editor.org/info/rfc3987>.

[RFC4648]

Josefsson, S., “The Base16, Base32, and Base64 Data Encodings”, RFC 4648, DOI 10.17487/RFC4648, October 2006. <https://www.rfc-editor.org/info/rfc4648>.

[RFC5646]

Phillips, A., Ed., and M. Davis, Ed., “Tags for Identifying Languages”, BCP 47, RFC 5646, DOI 10.17487/RFC5646, September 2009. <https://www.rfc-editor.org/info/rfc5646>.

[RFC7493]

Bray, T., Ed., “The I-JSON Message Format”, RFC 7493, DOI 10.17487/RFC7493, March 2015. <https://www.rfc-editor.org/info/rfc7493>.

[RFC7946]

Butler, H., Daly, M., Doyle, A., Gillies, S., Hagen, S., and T. Schaub, “The GeoJSON Format”, RFC 7946, DOI 10.17487/RFC7946, August 2016. <https://www.rfc-editor.org/info/rfc7946>.

[RFC8174]

Leiba, B., “Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words”, BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017. <https://www.rfc-editor.org/info/rfc8174>.

[RFC8259]

Bray, T., Ed., “The JavaScript Object Notation (JSON) Data Interchange Format”, STD 90, RFC 8259, DOI 10.17487/RFC8259, December 2017. <https://www.rfc-editor.org/info/rfc8259>.

A.2 Informative References**[OData-Aggregation]**

OData Extension for Data Aggregation Version 4.0.

See link in “[Related work](#)” section on cover page.

[ECMAScript]

ECMAScript 2023 Language Specification, 14th Edition, June 2023. Standard ECMA-262. <https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>.

Appendix B. Safety, Security and Privacy Considerations

This specification raises no security issues.

This section is provided as a service to the application developers, information providers, and users of OData version 4.0 giving some references to starting points for securing OData services as specified. OData is a REST-full multi-format service that depends on other services and thus inherits both sides of the coin, security enhancements and concerns alike from the latter.

For JSON-relevant security implications please cf. at least the relevant subsections of [\[RFC8259\]](#) as starting point.

Appendix C. Acknowledgments

C.1 Special Thanks

The contributions of the OASIS OData Technical Committee members, enumerated in [\[OData-Protocol, section C.2\]](#), are gratefully acknowledged.

C.2 Participants

OData TC Members:

First Name	Last Name	Company
George	Ericson	Dell
Hubert	Heijkers	IBM
Ling	Jin	IBM
Stefan	Hagen	Individual
John	Gathogo	Microsoft
Michael	Pizzo	Microsoft
Olga	Podolyako	Microsoft
Christof	Sprenger	Microsoft
Ralf	Handl	SAP SE
Gerald	Krause	SAP SE
Heiko	Theißen	SAP SE

Appendix D. Revision History

Revision	Date	Editor	Changes Made
Working Draft 01	2018-01-29	Hubert Heijkers	Initial proposal for a compact JSON format for OData Version 4.0
Working Draft 02	2026-08-20	Hubert Heijkers	Restructured as a normative specification for Version 4.02; extended to request payloads and delta payloads; introduced the wrapper object and the superset principle

Appendix E. Notices

Copyright © OASIS Open 2026. All Rights Reserved.

All capitalized terms in the following text have the meanings assigned to them in the OASIS Intellectual Property Rights Policy (the “OASIS IPR Policy”). The full [Policy](#) may be found at the OASIS website.

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published, and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this section are included on all such copies and derivative works. However, this document itself may not be modified in any way, including by removing the copyright notice or references to OASIS, except as needed for the purpose of developing any document or deliverable produced by an OASIS Technical Committee (in which case the rules applicable to copyrights, as set forth in the OASIS IPR Policy, must be followed) or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by OASIS or its successors or assigns.

This document and the information contained herein is provided on an “AS IS” basis and OASIS DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

As stated in the OASIS IPR Policy, the following three paragraphs in brackets apply to OASIS Standards Final Deliverable documents (Committee Specification, Candidate OASIS Standard, OASIS Standard, or Approved Errata).

[OASIS requests that any OASIS Party or any other party that believes it has patent claims that would necessarily be infringed by implementations of this OASIS Standards Final Deliverable, to notify OASIS TC Administrator and provide an indication of its willingness to grant patent licenses to such patent claims in a manner consistent with the IPR Mode of the OASIS Technical Committee that produced this deliverable.]

[OASIS invites any party to contact the OASIS TC Administrator if it is aware of a claim of ownership of any patent claims that would necessarily be infringed by implementations of this OASIS Standards Final Deliverable by a patent holder that is not willing to provide a license to such patent claims in a manner consistent with the IPR Mode of the OASIS Technical Committee that produced this OASIS Standards Final Deliverable. OASIS may include such claims on its website, but disclaims any obligation to do so.]

[OASIS takes no position regarding the validity or scope of any intellectual property or other rights that might be claimed to pertain to the implementation or use of the technology described in this OASIS Standards Final Deliverable or the extent to which any license under such rights might or might not be available; neither does it represent that it has made any effort to identify any such rights. Information on OASIS' procedures with respect to rights in any document or deliverable produced by an OASIS Technical Committee can be found on the OASIS website. Copies of claims of rights made available for publication and any assurances of licenses to be made available, or the result of an attempt made to obtain a general license or permission for the use of such proprietary rights by implementers or users of this OASIS Standards Final Deliverable, can be obtained from the OASIS TC Administrator. OASIS makes no representation that any information or list of intellectual property rights will at any time be complete, or that any claims in such list are, in fact, Essential Claims.]

The name “OASIS” is a trademark of [OASIS](#), the owner and developer of this specification, and should be used only to refer to the organization and its official outputs. OASIS welcomes reference to, and implementation and use of, specifications, while reserving the right to enforce its marks against misleading uses. Please see <https://www.oasis-open.org/policies-guidelines/trademark/> for above guidance.